Project 1 – Parameter Estimation

Wesley Ricaporte, Nicholas T.

Dr. Talles Santos

ECEN4632 – DSP

9/29/2026

0. Table of Contents

1. Introduction 4
2. Original QAM Method 5
   2.1 Overview 5
   2.2 Algorithm 5
      2.2.1 Error Matrix 5
      2.2.2 Pseudo-Inverse 5
      2.2.2 Parameter Vector S 6
      2.2.4 Parameter Extraction 6
   2.3 Comments 6
3. Experimental FFT Method 7
   3.1 Overview 7
   3.2 Algorithm 7
      3.2.1 FFT Dominant Frequency Detection 7
      3.2.2 Band-Pass Filtering 8
      3.2.3 Zero-Crossing Detection 8
      3.2.4 Zero-Crossing Distancing 8
      3.2.5 Frequency Aggregation 9
      3.2.6 QAM 9
4. Experimental Setup 10
   4.1 Materials 10
   4.2 Electrical Configuration 10
   4.3 Procedure 10
5. Results 12
6. Analysis 16
7. Conclusion 17
A. Code Setup 18
   A.1 Technology Stack 18
   A.2 Components 18

A.2.1 Waveforms Script 18
A.2.1.1 Data Acquisition Loop 18
A.2.1.2 Mailboxes 19
A.2.2 Server 20
A.2.2.1 Support Loop 20
A.2.2.2 Processing 21
A.2.3 Client 21
A.3 Code Execution 21
B. Code 23
C. References 23

1. Introduction

Pure sinusoidal functions can be described using exactly four parameters: amplitude (*A*), frequency (*f*), phase shift (ϕ), and offset (*C*), as in equation 1.1 below:

$$x(t) = Acos(2\pi ft + \phi) + C \tag{1.1}$$

Generating signals is clearly trivialized by this formula. However reversing the process–estimating these parameters from a set of points–is far more difficult and far less clean. This paper introduces a novel method to estimate these parameters by adjusting a preexisting method. The accuracy and consistency of the estimated parameters are evaluated by comparing the error between the original and estimated signals using the mean square error and standard deviation.

2. Original QAM Method

2.1 Overview

Quadrature Amplitude Modulation, or QAM, is a method that uses orthogonal sinusoidal waves to estimate the amplitude, phase shift, and DC offset of some original sinusoidal signal. This method assumes that all parameters of the original sinusoidal signal are constant across sampled period. Note that the desired frequency of the parameterized signal must be provided for this method to function.

2.2 Algorithm

This method is fairly simple to implement. For input signal row vector *X*, consisting of *n* samples sampled at a rate of f_s:

1. Compute the *Error* matrix (*E*)
2. Compute the pseudo-inverse of *E*, producing E_p
3. Compute the product XE_p, producing the column vector *S*
4. Extract the relevant parameters from *S*, yielding *A*, ϕ, and *C*

The details of this algorithm are expanded on in the subsections below.

2.2.1 Error Matrix

E is a matrix of dimensions (*n* x 3), produced by concatenating three separate column vectors. Equations *2.1, 2.2, 2.3* respectively describe columns 1, 2, 3 for each row t_n, $t_n \in \{0, 1, 2, \dots, n-1\}$,

$$\sin(2\pi f_s t_n) \quad (2.1)$$

$$\cos(2\pi f_s t_n) \quad (2.2)$$

$$1 \quad (2.3)$$

Note that column 3 consists purely of the constant value 1 and does *not* vary with the row.

2.2.2 Pseudo-Inverse

Only square matrices with non-zero determinants are invertible. The QAM method relies on obtaining the inverse of the error matrix, which is clearly not guaranteed to be square. As such, we instead compute the *Moore-Penrose* inverse, as shown in equation *2.4* (Weisstein, 2026), for matrix *A* and transposed matrix *A'*.

$$(A'A)^{-1}A' \quad (2.4)$$

Matrix E_p is thus calculated using (2.4), as $E_p = (E'E)^{-1}E'$.

2.2.2 Parameter Vector S

The *S* vector is computed by treating the original signal *X* as a (1 x *n*) matrix, then multiplying by the (*n* x 3) E_p matrix, creating the final (1 x 3) matrix *S* which can be interpreted as a 3-dimensional vector.

2.2.4 Parameter Extraction

The final 3-dimensional vector *S* can be interpreted as a row vector with the following entries: $[\ \alpha, \beta, C\ ]$. The parameters *α* and *β* can be used directly in equation *2.5* to describe the sinusoid. To fit (*1.1*), these parameters can be transformed into *A* and *ϕ* using equations *2.6* and *2.7* (Notice that these equations effectively transform from cartesian to polar values).

$$\alpha sin(2\pi ft) + \beta \cos(2\pi ft) + C \quad (2.5)$$

$$A = \sqrt{\alpha^2 + \beta^2} \quad (2.6)$$

$$\phi = tan^{-1}(\frac{\alpha}{\beta}) \quad (2.7)$$

2.3 Comments

For a given known constant frequency *f*, this equation gives the best possible parameters *A, ϕ, C* possible for fitting the original equation, minimizing error. This algorithm's main disadvantage is its reliance on an already-known frequency *f*, a hole that the proposed method aims to fill.

3. Experimental FFT Method

3.1 Overview

Our technique augments the original QAM method through adding *automatic frequency detection* using a Fast Fourier Transform (FFT)-based approach. This detected frequency is then used as the *f* in the QAM method, which is then run unmodified.

3.2 Algorithm

Our experimental method processes the sampled signal in a fixed pipeline; first computing the estimated frequency, then estimating the amplitude, phase, and offset using the proven QAM method. For input signal row vector X, consisting of n samples sampled at a rate of f_s:

1. FFT Dominant Frequency Detection
2. Band-Pass Filtering
3. Zero-Crossing Detection
4. Zero-Crossing Distancing
5. Frequency Aggregation
6. QAM

The details of this algorithm are expanded on in the subsections below.

3.2.1 FFT Dominant Frequency Detection

This step seeds the rest of the algorithm with an initial (imprecise) frequency guess. This is achieved through the following procedure:

1. Transform input signal X into the frequency domain using the Fast Fourier Transform (FFT), producing X_s
2. Find the sample within the first half[1] of X_s with the greatest magnitude[2]. This represents the "dominant" frequency component. Call the sample index i, $i \in \{0, 1, \dots, n-1\}$
3. Compute the initial guess f_i using equation (3.1)

$$f_0 = i\left(\frac{f_s}{n}\right) \tag{3.1}$$

[1] Note that only the first half is searched due to the periodic nature of the output of the FFT. The second half of the FFT output can be viewed as a mirrored version of the first half and thus has no value.
[2] Note that index 0 is ignored, to avoid conflating the DC offset as a signal frequency component

Note that this equation is a simple linear interpolation, mapping from $\{0, 1, \dots, n\}$ *to* $0, 1, \dots, f_s$.

Notice that this algorithm *intentionally* ignores any and all information stored in all X_s buckets besides the largest.

3.2.2 Band-Pass Filtering

The initial guess, f_i is used to generate a filtered signal X_f. This computation is performed via the following procedure:

1. Compute a window W around f_i, such that the total width of the window takes up nine frequency buckets in X_s
2. Apply W to X_s, such that all samples within the window are untouched, and all samples outside the window are reduced to zero
3. Compute the *Inverse* Fast Fourier Transform (IFFT) of X_s to produce X_f

The end result of this process is the clean time-domain signal X_f, representing X after being run through an ideal band-pass filter to largely remove high-frequency noise and low-frequency undulations.

3.2.3 Zero-Crossing Detection

This step creates a list Z, $\|Z\| \leq \frac{n}{2}$, where each entry $Z[i]$ represents some unique point $X_f[k] \in X_f$ where each $Z[i]$ represents some unique $X_f[i]$ which satisfies equation (3.2).

$$X_f[k] \leq 0 \wedge X_f[k+1] > 0 \tag{3.2}$$

The value stored in $Z[i]$ is the approximated zero-crossing location using linear interpolation between points $X_f[k]$, $X_f[k+1]$. Note that the values in $Z[i]$ are stored as strictly increasing scalars, for ease of future processing.

3.2.4 Zero-Crossing Distancing

From Z, compute Z_d, where $Z_d[i]$ represents the distance between subsequent zero crossings, using equation (3.3). Notice that $\|Z_d\| = \|Z\| - 1$. We assume there are sufficiently many samples X_f such that $\|Z\| \geq 2$.

$$Z_d[i] = Z[i+1] - Z[i] \tag{3.3}$$

3.2.5 Frequency Aggregation

This step exists to aggregate the list of distances Z_d into a single scalar value, which can then be treated as the dominant frequency of X. This is achieved through simply averaging all values in Z_d, as is shown in equation (3.4), to yield T_z.

$$T_Z = \frac{\sum Z_d}{\|Z_d\|} \tag{3.4}$$

T_z represents the period between upward zero crossings. As each cycle of the signal represented by X *contains exactly one* upward zero crossing, $T_z = T$, the period of the signal. It is trivial to show then that the frequency f can be computed through equation (3.5).

$$f = \frac{1}{T_Z} \tag{3.5}$$

3.2.6 QAM

The computed frequency f, along with the original signal X, are plugged into the QAM method previously described in section 2, yielding the estimated A, ϕ, and C.

4. Experimental Setup

This section describes the methods used to compare the proposed FFT-based approach with the original (control) QAM approach.

These tests were performed using real signal generators and ADCs, to accurately capture real-world noise.

4.1 Materials

- Analog Discovery 2 (AD2)
- Rohde & Schwarz RTB2004

4.2 Electrical Configuration

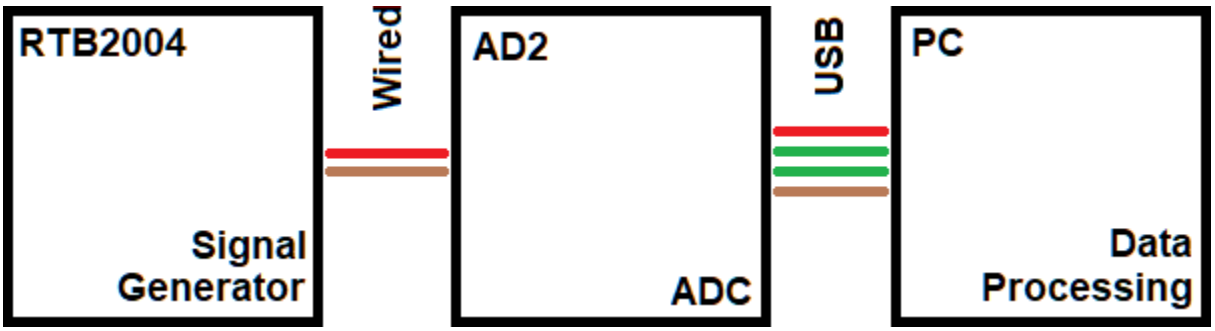


Figure 4.1a: The electrical setup used to generate the results shown in section 5

The signal generator of the RTB2004 was attached to analog channel 1 of the AD2. The AD2 was then connected to a computer running *Waveforms*, sending the signal data to our processing and visualization code.

4.3 Procedure

To test the proposed FFT-based approach against the original QAM approach, multiple signal frequencies with various amounts of noise were tested. Between tests, no changes were made to *any* code[3]. The samples listed in *Table 4.3a* were tested. The original QAM approach was run with an expectation of 1.0kHz.

[3] This is especially relevant for the original QAM strategy, as it relies on knowing the actual frequency.

Table 4.3a: The sampled signals tested against, as generated by the RTB2004

Frequency	**Amplitude (V_p)**	**DC Offset (V)**	**Noise (V_p)**
1.0 kHz	1	4	0.1
1.1 kHz	1	4	0.1
900 Hz	1	4	0.1
0.8 kHz – 1.2 kHz *SWEEP*	1	4	0.1

The results from these tests are available in *Section 5: Results*.

5. Results

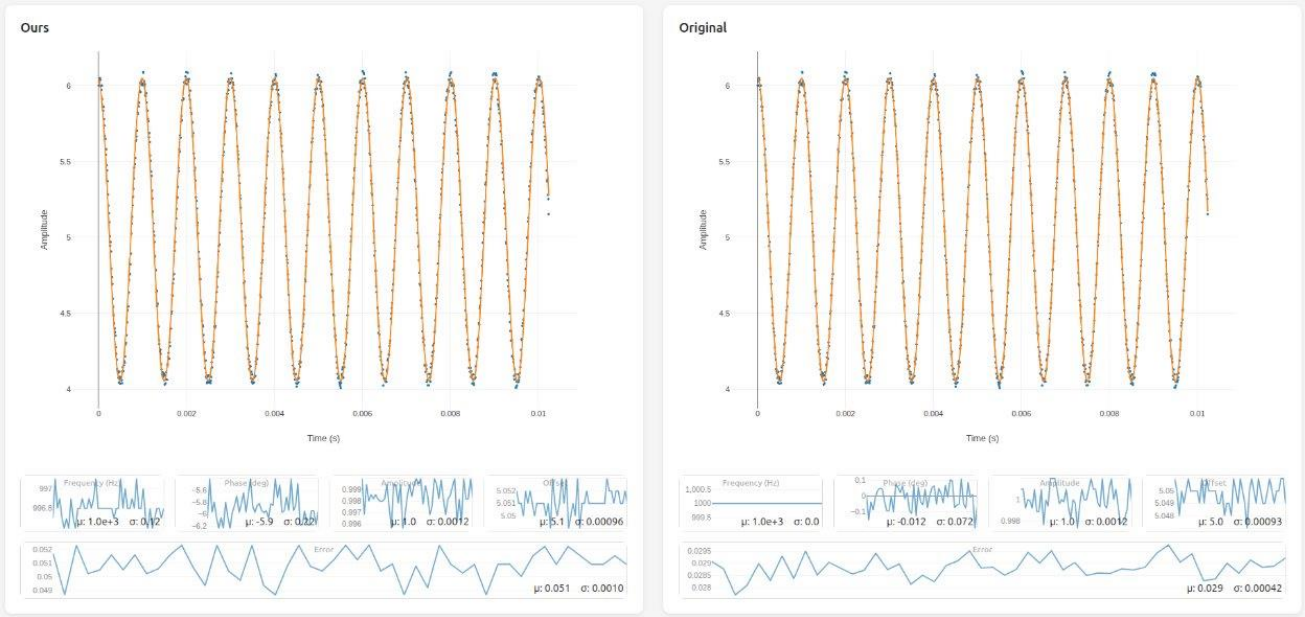


Figure 5.1: Final iteration results for a 1kHz sinusoidal input with 10% added noise, showing the estimated and original signals, along with the mean (μ) and standard deviation (σ) for frequency, phase, amplitude, offset, and error.

Table 5.1: Error comparison between the original and final algorithms for a 1 kHz sinusoidal input with 10% added noise.

Algorithm	**Mean Error (μ)**	**Standard Deviation (σ)**
Original Algorithm	0.029	0.00042
Final Algorithm	0.051	0.0010

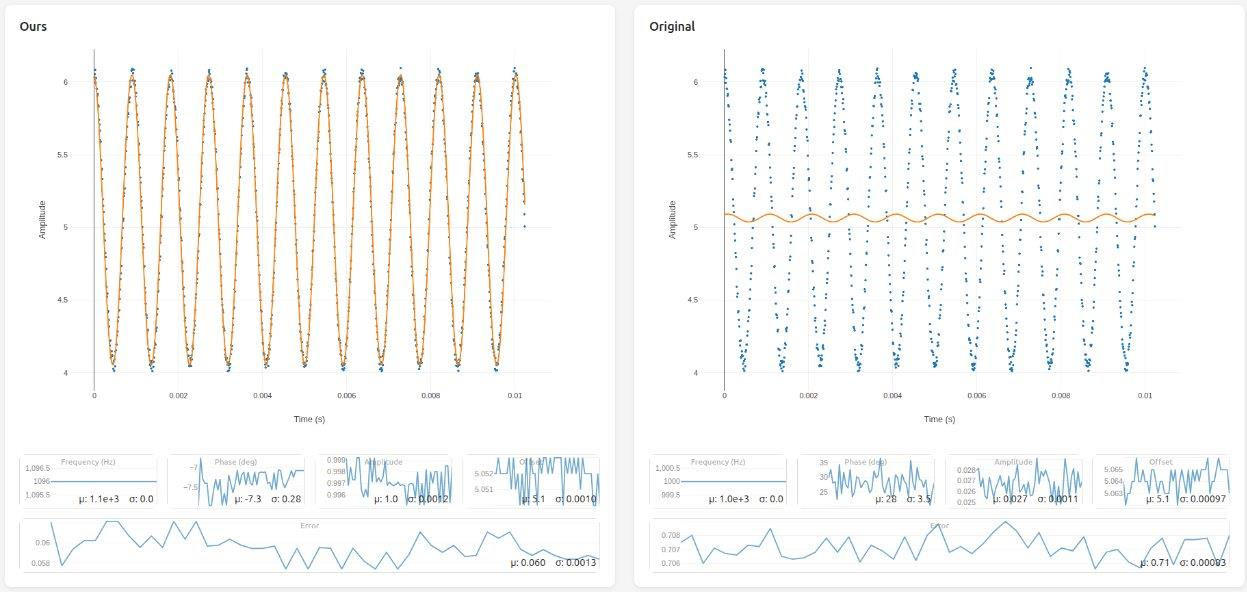


Figure 5.2: Final iteration results for a 1.1kHz sinusoidal input with 10% added noise, showing the estimated and original signals, along with the mean (μ) and standard deviation (σ) for frequency, phase, amplitude, offset, and error.

Table 5.2: Error comparison between the original and final algorithms for a 1.1 kHz sinusoidal input with 10% added noise.

Algorithm	Mean Error (μ)	Standard Deviation (σ)
Original Algorithm	0.71	0.00093
Final Algorithm	0.060	0.0013

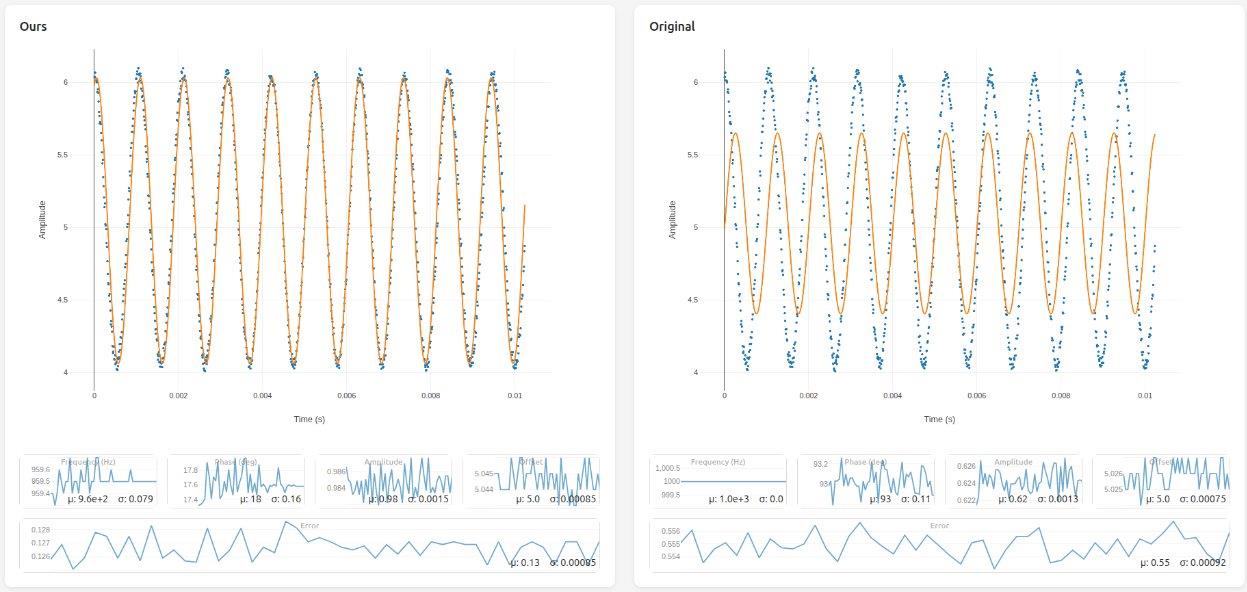


Figure 5.3: Final iteration results for a 900 Hz sinusoidal input with 10% added noise, showing the estimated and original signals, along with the mean (μ) and standard deviation (σ) for frequency, phase, amplitude, offset, and error.

Table 5.3: Error comparison between the original and final algorithms for a 900 Hz sinusoidal input with 10% added noise.

Algorithm	Mean Error (μ)	Standard Deviation (σ)
Original Algorithm	0.55	0.00092
Final Algorithm	0.13	0.00085

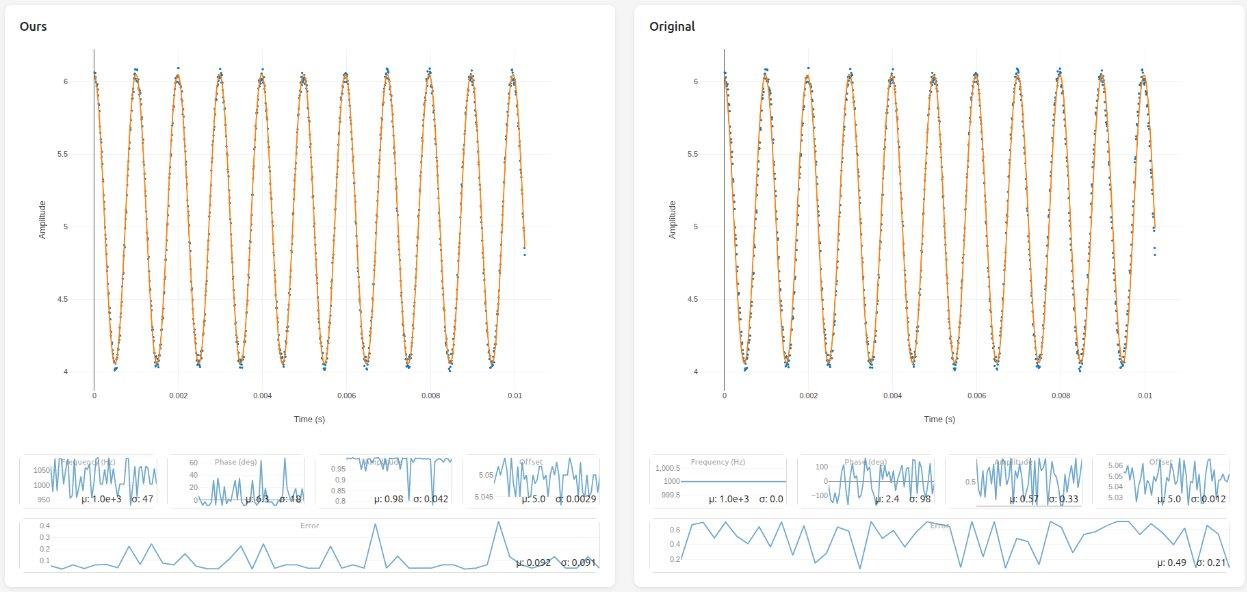


Figure 5.4: Final iteration results for a 0.8 kHz to 1.2 kHz frequency sweep with 10% added noise, showing the estimated and original signals, along with the mean (μ) and standard deviation (σ) for frequency, phase, amplitude, offset, and error.

Table 5.4: Error comparison between the original and final algorithms for a 0.8 kHz to 1.2 kHz frequency sweep *with 10% added noise.*

Algorithm	Mean Error (μ)	Standard Deviation (σ)
Original Algorithm	0.49	0.21
Final Algorithm	0.092	0.0091

Figures 5.1 through 5.4 show the estimated signal outputs of the final algorithm compared with the original QAM method for each test case. At 1 kHz, the original algorithm produced a mean error of 0.029 compared with 0.051 for the final algorithm. The original algorithm also had a lower standard deviation at this frequency (Table 5.1). At 1.1 kHz, the original algorithm's mean error increased to 0.71, while the final algorithm produced a mean error of 0.060 (Table 5.2). At 900 Hz, the original algorithm produced a mean error of 0.55 compared with 0.13 for the final algorithm (Table 5.3). During the 0.8 to 1.2 kHz frequency sweep, the original algorithm produced a mean error of 0.49 with a standard deviation of 0.21, while the final algorithm produced a mean error of 0.092 with a standard deviation of 0.0091 (Table 5.4).

6. Analysis

The results show a clear difference between the original QAM method and the final method. At the 1 kHz test frequency, the original algorithm had a lower mean error and standard deviation than the final method. This result was expected because the original QAM method was given the correct frequency of 1 kHz. When the input signal matches this value, the original method has the advantage of already knowing the signal frequency.

The final method's advantage became apparent when the input frequency was changed. At 1.1 kHz, the mean square error of the original algorithm went from 0.029 to 0.71, while the final algorithm stayed the same going from 0.051 to 0.060. At 900 Hz, the mean error of the original algorithm increased from 0.029 to 0.55, while the final algorithm increased from 0.051 to 0.13. Although the final method showed a larger error than at 1 kHz, it remained much closer to the original signal than the original QAM method.

The frequency sweep from 0.8 kHz to 1.2 kHz provided a realistic comparison between the two methods. The original algorithm produced a mean error of 0.49 with a standard deviation of 0.21. In comparison, the final algorithm produced a mean error of 0.092 with a standard deviation of 0.0091. The much smaller standard deviation also shows that the final method remained more consistent as the input frequency changed.

The square mean error is the average square difference between the estimated and original signal. A lower mean error means that the algorithm’s estimate was closer to the original signal. The standard deviation shows how the error varied throughout the test. A smaller standard deviation means the error stayed more consistent, and a larger standard deviation means the error changed more from one measurement to another. The mean and standard deviation show the accuracy and consistency of the algorithms as the input frequency is changed.

Overall, the final algorithm did not improve the results when the exact frequency was known. Its advantage was estimating the frequency from the sampled signal. The FFT allowed for an initial estimate of the dominant frequency, and the filtering, along with the zero-crossing detection, computed an estimated frequency before the QAM calculations. This made it so the final algorithm could estimate signal parameters when the correct frequency is not known beforehand.

7. Conclusion

For situations where the frequency of the original signal is unknown, the proposed FFT-based approach works fairly well, automatically computing an estimated frequency value. However this value is never exact, and because the underlying QAM method is so sensitive to the frequency of this signal, the error can vary dramatically depending on the accuracy of the initial frequency estimation. This was shown in the 1.1 kHz, 900 Hz, and frequency-sweep tests, where the final method produced a lower mean error than the original QAM method using an incorrect assumed frequency. The frequency sweep also showed a much lower standard deviation, meaning it performed more consistently as the input frequency changed.

As such, the original QAM method is recommended whenever the original frequency is already known—it requires less processing power and tends to produce a result with lower error.

Overall, the proposed FFT-based approach fills the niche of parameter estimation where the signal frequency is either unstable or unknown, providing a better result than assuming an incorrect frequency.

A. Code Setup

This project was completed entirely without the use of MATLAB, to prove the viability and speed of other languages for heavy data processing. This appendix covers the implementation details of this approach.

A.1 Technology Stack

Data acquisition was performed using *Waveform*'s built-in scripting language. All data processing in this project was performed in *Python* using the *NumPy* library. Data rendering was performed in a separate process running in a browser, rendering a webpage powered by *Vite*, using the *Plotly.js* plotting library.

Communication between all components was facilitated using either the filesystem or *WebSockets*.

A.2 Components

This project's code is cleanly divisible into three major components. These subsections go into further detail as to how each component works.

A.2.1 Waveforms Script

The code for this section is available in the GitHub repository under the `waveforms/src/` folder.

Waveforms is the software used to communicate with the physical AD2 hardware. It accepts custom scripts running code adhering to the EcmaScript 5 (ES5) standard.

Code for waveforms was written in Typescript, a language that can be configured to compile into ES5. This was done for ease-of-programming and readability. Typescript, as the name suggests, adds *types* to EcmaScript. This helps the language to catch errors that the programmer may otherwise have missed.

The *Waveforms* script performs two tasks:

1. Gathering data from the AD2
2. Shipping data from *Waveforms* to the next component in the data-processing pipeline

A.2.1.1 Data Acquisition Loop

This loop lives under the `*waveforms/src/main.ts*` file in the *GitHub* repository, within the *main.ts* file. This code performs the following sequence:

1. Ensure the *Scope* tab is open in *Waveforms*. Without this, the script is unable to access AD2 channel data
2. Initialize a send-only double-buffer *Mailbox* (see section A.2.1.2 for more details)
3. Wait for a new sample to come in
    a. *Sample is ignored if generated too quickly. This is done to limit the number of* Mailbox *writes to a reasonable frequency*
4. Read scope data. This includes the raw data (points in an ES5 array), sampling rate (for later real-world frequency conversion), and initial sample time (to properly respect phase shift throughout the process)
5. Generate an in-memory CSV file, where columns are in the format [*time (ms)*, *data (V)*]
6. Write the contents of the in-memory CSV file into the *Mailbox*
7. *Main processing loop done;* REPEAT FROM STEP 3

A.2.1.2 Mailboxes

A *Mailbox* is a protocol which uses the filesystem to transfer discrete units of data between processes. To avoid wearing out real disk drives, it is recommended that all *Mailboxes* are pointed to a tmpfs file system partition[4].

The code for the *Mailbox* used in this project can be viewed on the *GitHub* repository under the `waveforms/src/uMail.ts` directory. The *UMail2Tx* class is the only Mailbox in use. The previous is a simpler version of the same overall concept.

The name *UMail2Tx* fully describes the *Mailbox*'s intended operation.

- **U**: Unchecked. Data is sent without receipt ever being verified
- **Mail**: Mailbox
- **2**: The number of buffers to write to
- **Tx**: The *Mailbox* only supports transmission

This *Mailbox* works off of a double-buffering system, making use of 3 files:

- *t_gen.mbx*: A generation counter stored in plain ASCII. Used both to point at the "readable" mailbox file and to indicate when a new message in the mailbox is ready
- *t_dat0.mbx*: One of the buffers in the *Mailbox*. Readable by the *RX* side iff the generation counter is *even*

[4] This is a part of the file system that runs purely in-memory, and never actually writes to the computer's real disk. As such, it is generally much faster than the traditional filesystem, at the cost of not persisting data. Modern Linux distributions generally have a tmpfs filesystem mounted under the `/tmp/` directory

- *dat1.mbx*: One of the buffers in the *Mailbox*. Readable by the *RX* side iff the generation counter is *odd*

The *UMail2Tx* protocol requires one piece of synchronized data between the sender and receiver: the *generation counter*. This is a number that is incremented whenever a new message is sent. The act of this value changing indicates to the receiver that a new message is available from the sender. The value after changing indicates which buffer to read from.

The procedure to write data to a *UMail2Tx* is as follows:

1. Identify the currently non-readable file (based on the current generation counter value)
2. Write the data to the non-readable file, overwriting any previous contents
3. Increment the generation counter by 1. This points the reader to the file we just wrote

In this way, *Mailboxes* are able to effectively transmit infinitely many discrete packets of information while taking up only 3 files on the disk at any given time.

A.2.2 Server

The code for this section is available in the GitHub repository under the `server/` folder.

The server itself runs on the same machine running the *Waveforms* process. It is broken into two major parts, one driving the other:

- Support Loop
- Processing

The details of each part are discussed below.

A.2.2.1 Support Loop

This loop handles reading the *UMail2Tx Mailbox* from *Waveforms*, converting the raw string data into a *NumPy ndarray*, then handing the *ndarray* over to the processing code (See *Section A.2.2.2*). The processing section returns an *CosDesc*—a complete description of a sinusoidal wave that the processing code believed best fit the input *ndarray* samples. This description is then used to synthesize a cosine wave.

The synthesized wave is then compared against the original, through the Mean-Square-Error metric. This error metric, the original and synthesized signals, and the description of the cosine wave, is then sent via *WebSockets*[5] to the client.

A.2.2.2 Processing

This portion of the code can be found under `server/process.py`, and exports two main functions: `processA` and `processB`. These functions each independently process the passed in ndarray representing the original signal, and return a *CosDesc*, a complete description of a sinusoidal that the processing code determined best fits the input data. This description is made of the following scalars:

- Amplitude
- Offset
- Phase
- Frequency

The code for the CosDesc *data class can be found under `server/cosdesc.py`.*

A.2.3 Client

The code for this section is available in the GitHub repository under the `client/` folder.

This component is the simplest of the three. Its job is to simply render the data provided by the *WebSocket* connection to the server and render the data to the screen.

Graph rendering is handled by *Plotly.js*, a plotting library chosen specifically due to its ability to plot thousands of points without majorly slowing down the browser.

Client code is rendered using the *Vite* framework. This provides many advantages, the major being provisions for writing browser-based apps in TypeScript, rather than needing to use raw JavaScript.

A.3 Code Execution

This project requires the following programs to be installed on your computer:

- node.js (Install at https://nodejs.org/en/download)
- python3 (Install at https://www.python.org/downloads/)

If they are not installed this project will not function.

[5] A *WebSocket* is a formal standard of sending data between multiple different processes running on the internet, allowing data acquisition to occur on one machine, and rendering/visualization to occur on another in real time

Additionally this project expects to run in a POSIX environment (Linux and MacOS users can run the project natively. Windows users will need to install Windows Subsystem for Linux).

After downloading the project, one must first install all node packages. Run command `npm i` in the project's root directory. Once this successfully completes, the project is ready to run.

Follow the steps below to run the project:

1. Run the command `npm run build` from the project's root directory. This will produce the build artifact `waveforms/build/build.js`, representing the ES5 *Waveforms* script. When built properly, this file is a single line of compressed EcmaScript.
2. Open *Waveforms* and copy the contents of the built `waveforms/build/build.js` into the *Script* tab. Also open the *Scope* tab
3. Connect the AD2 to your computer and *Waveforms*, then start the *Waveforms* script
4. In the project root directory, run the command `npm run server`. This will start the python server. If it is your first time running this command, it will take some time to set up the python virtual environment and install all packages
5. In the project root directory, run the command `npm run client`. In the terminal, this will show a URL (`localhost:5173` or `127.0.0.1:5173`). Navigate to this URL in your preferred web browser[6]

[6] For performance reasons, the client only functions on browsers that support WebGL. If your browser does not support this, a message indicating as much will be displayed in the main graphs

B. Code

All code for this project is publicly available on GitHub at https://github.com/WesleyRicaporte/Project_1_DSP

C. References

Weisstein, E. (2026, September 2). *Moore-Penrose Matrix Inverse*. Retrieved from Wolfram MathWorld: https://mathworld.wolfram.com/Moore-PenroseMatrixInverse.html